

UNDERSTANDING
DNS

Domain Name System — The Complete Technical Guide

Brij Mohan

Network Engineer & Web Technologist

■ 25 Pages

■ 12 Chapters

■ Deep Dive

■ Security Focus

■ History

■ Architecture

■ Record Types

■ Resolution

■ Security

■ DNSSEC

■ DoH/DoT

■ Troubleshoot

Network Fundamentals Series • Edition 2025

CH
0

Table of Contents

01 History & Origins of DNS →

- ARPANET era
- Paul Mockapetris 1983
- HOSTS.TXT evolution

02 DNS Architecture & Hierarchy →

- Root servers
- TLD servers
- Authoritative nameservers

03 How DNS Resolution Works →

- Recursive vs Iterative
- Full resolution walkthrough
- Caching layers

04 DNS Record Types — Complete Guide →

- A, AAAA, CNAME, MX, TXT, NS
- SOA, PTR, SRV, CAA, NAPTR
- Wildcard records

05 TTL — Time To Live →

- TTL mechanics
- Propagation
- Best practices

06 DNS Message Format (RFC 1035) →

- Header structure
- Question section
- Answer section

07 DNS Zones & Zone Transfers →

- Zone files
- Primary vs Secondary
- AXFR / IXFR

08 Public DNS Resolvers →

- Google 8.8.8.8
- Cloudflare 1.1.1.1
- OpenDNS
- Benchmarks

09 DNS Security — Attacks →

- Cache poisoning
- DNS hijacking
- DDoS amplification

10 DNSSEC — DNS Security Extensions [→](#)

- Chain of trust
- KSK/ZSK
- RRSIG & DS records

11 Modern DNS — DoH, DoT, DoQ [→](#)

- DNS over HTTPS
- DNS over TLS
- DNS over QUIC

12 Troubleshooting & Tools [→](#)

- nslookup, dig, host
- Wireshark DNS
- Common issues

CH
1

History & Origins of DNS

The story of DNS begins long before the World Wide Web. In the 1960s and early 1970s, the **ARPANET** — the precursor to today's Internet — connected only a handful of computers. Naming was trivially simple: a single text file called **HOSTS.TXT** was maintained at the Stanford Research Institute (SRI) and manually distributed to every connected host.

By 1982, ARPANET had grown to hundreds of hosts. The HOSTS.TXT file had become a maintenance nightmare — it was updated twice a week, downloads consumed significant bandwidth, and name collisions were a constant problem. Something fundamentally new was needed.

The Birth of DNS — 1983

In November 1983, **Paul Mockapetris** published RFC 882 and RFC 883, describing the Domain Name System. The design was revolutionary: instead of a single central file, DNS distributed the naming database across thousands of servers in a hierarchical tree structure. No single machine needed to know all names — each zone only needed to know about its own portion of the namespace, and how to refer queries upward.

Year	Event
1969	ARPANET launched — 4 connected hosts
1971	HOSTS.TXT introduced at SRI-NIC
1982	HOSTS.TXT grows unmanageable — 400+ hosts
1983	Paul Mockapetris publishes RFC 882/883 — DNS born
1984	First DNS implementation: JEEVES (TOPS-20)
1985	BIND (Berkeley Internet Name Domain) released — still dominant today
1987	RFC 1034/1035 — still the core DNS standard
1995	Internet explosion — DNS handles millions of names
1997	RFC 2136 — Dynamic DNS (DDNS)
1999	DNSSEC first proposed (RFC 2535)
2008	Kaminsky attack exposes DNS cache poisoning
2010	Root zone signed with DNSSEC
2016	Mirai botnet DDoS attack on Dyn DNS — major outage
2018	RFC 8484 — DNS over HTTPS (DoH) standardised

2019	Cloudflare 1.1.1.1 popularises privacy-first DNS
2022	QUIC-based DNS (DoQ) gains traction

DID YOU KNOW

Paul Mockapetris was awarded the IEEE Internet Award in 2003 for his invention of DNS. He said: 'I designed DNS to be simple, scalable, and decentralised. I never imagined it would underpin a multi-trillion-dollar global economy four decades later.'

CH
1

■ History & Origins (continued)

BIND — The Reference Implementation

BIND (Berkeley Internet Name Domain) was written in 1984 by four UC Berkeley graduate students as part of their BSD Unix project. It became — and remains — the most widely deployed DNS software in the world, running a significant portion of the Internet's authoritative and recursive nameservers. BIND is maintained by the Internet Systems Consortium (ISC) and is open-source.

The Kaminsky Attack — 2008

In 2008, security researcher **Dan Kaminsky** discovered a fundamental flaw in DNS that allowed attackers to poison resolver caches much faster than previously thought — injecting forged DNS responses and redirecting users to malicious sites. The coordinated industry response — patching all major resolvers within weeks — is considered one of the most successful security coordinations in internet history. This attack accelerated the adoption of DNSSEC.

The Dyn DDoS Attack — 2016

On October 21, 2016, the **Mirai botnet** — composed of hundreds of thousands of compromised IoT devices — launched a massive Distributed Denial of Service (DDoS) attack against Dyn, a major DNS provider. The attack took down major websites including Twitter, Netflix, Reddit, CNN, and PayPal for hours, demonstrating how critical — and fragile — DNS infrastructure can be.

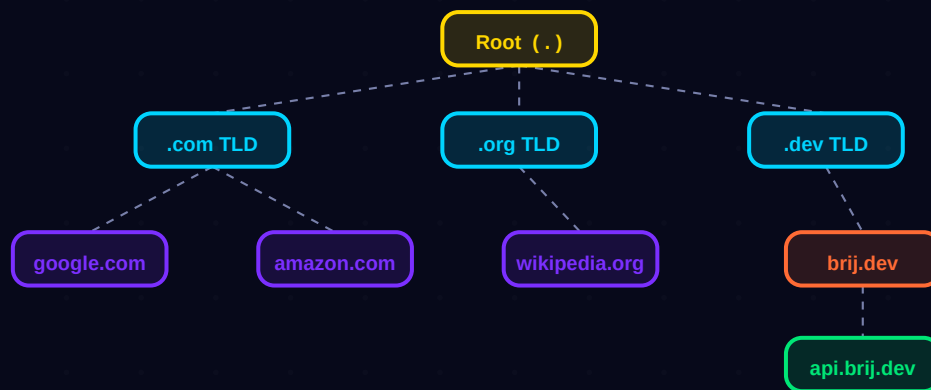
WARNING

The 2016 Dyn attack peaked at over 1.2 Tbps of traffic — the largest DDoS ever recorded at that time. It exposed the danger of centralising DNS infrastructure and led to widespread adoption of anycast routing and multi-provider DNS strategies.

CH
2

■ DNS Architecture & Hierarchy

DNS is a **distributed, hierarchical database**. Think of it as an upside-down tree: the root is at the top, and branches spread downward into top-level domains, second-level domains, subdomains, and hosts. Each level of the hierarchy is managed by different organisations — nobody owns the entire tree.



The Three Levels of DNS Servers

Level	Name	Count	Example	Managed By
0	Root Nameservers	13 sets (1000+ anycast)	a.root-servers.net	ICANN, Verisign, etc.
1	TLD Nameservers	Thousands	a.gtld-servers.net	Registry operators
2	Authoritative NS	Millions	ns1.cloudflare.com	Domain owners / registrars
3	Recursive Resolvers	Billions	8.8.8.8	ISPs, public DNS

Root Nameservers

There are **13 root nameserver identities** (labeled A through M), but these are distributed across over **1,600 physical servers** worldwide using **anycast routing** — where multiple machines share the same IP address and traffic is automatically routed to the closest one. Root servers only know where to find TLD nameservers; they don't store any domain records themselves.

Letter	Operator	IPv4 Address	Anycast Nodes
A	Verisign	198.41.0.4	6
B	USC-ISI	199.9.14.201	6
C	Cogent Comm.	192.33.4.12	8

D	UMD	199.7.91.13	53
E	NASA Ames	192.203.230.10	24
F	Internet Systems Consortium	192.5.5.241	283+
G	DISA / DoD	192.112.36.4	6
H	US Army Research Lab	198.97.190.53	2
I	Netnod (Sweden)	192.36.148.17	59
J	Verisign	192.58.128.30	119
K	RIPE NCC	193.0.14.129	73
L	ICANN	199.7.83.42	180+
M	WIDE Project (Japan)	202.12.27.33	10

CH
2

■ Architecture (continued)

TLD Nameservers

Top-Level Domain (TLD) nameservers are responsible for all domains under a specific TLD like [.com](#), [.org](#), [.dev](#), or country-code TLDs like [.in](#), [.uk](#). There are over 1,500 TLDs today. These servers only store NS (nameserver) records pointing to the authoritative nameservers for each registered domain.

TLD Type	Examples	Governed By	Count
Generic (gTLD)	.com .org .net .edu .gov .mil	ICANN	~1,200
New gTLD	.app .dev .cloud .shop .blog	ICANN (2012+)	~1,400+
Country Code (ccTLD)	.in .uk .de .jp .cn .ru .au	Each country	249
Sponsored (sTLD)	.gov .edu .mil .museum .aero	Sponsoring org	~15
Internationalised (IDN)	Arabic, Chinese, Cyrillic TLDs	ICANN	Growing

Authoritative Nameservers

Authoritative nameservers are the **final authority** for a domain's DNS records. When you register a domain and set up DNS hosting (via your registrar, Cloudflare, AWS Route 53, etc.), you configure at least two authoritative nameservers for redundancy. These servers hold the actual A records, MX records, CNAME records, and all other resource records for your domain.

The registrar communicates your nameserver addresses to the TLD registry, which stores an **NS record + glue record** (the IP of your NS) in the TLD zone. This is what makes the chain of delegation work.

CONCEPT: GLUE RECORDS

Glue records solve the chicken-and-egg problem: if ns1.brij.dev is the nameserver for brij.dev, how do you find ns1.brij.dev without first resolving brij.dev? The answer: the TLD registry stores the IP of ns1.brij.dev directly — this is the glue record.

CH

3

■

How DNS Resolution Works

DNS resolution is the process of translating a human-readable domain name into an IP address. This happens thousands of times per day for a typical user, usually invisibly and in under 50 milliseconds.

Recursive vs Iterative Resolution

Aspect	Recursive Resolution	Iterative Resolution
Who does work?	Resolver does all the work for the client	Client contacts each server directly
Burden	Heavy on resolver, light on client	Heavy on client
Used by	Client → Recursive resolver	Recursive resolver → other NS
Caching	Resolver caches for all its clients	Each client must cache independently
Security	Centralised — easier to add filtering/security	Distributed — harder to secure

The Full Resolution Walkthrough

Let's trace exactly what happens when you type <https://brij.dev> in your browser:

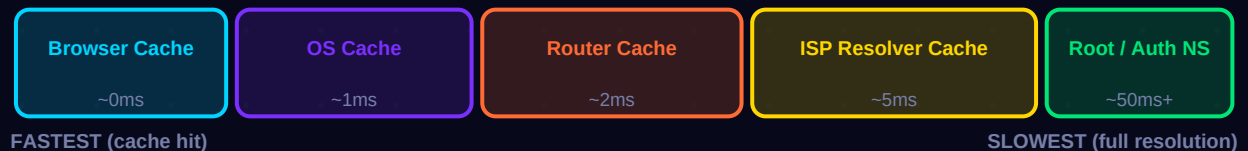
#	Who	What Happens	Cache?
1	Your Browser	Checks its own DNS cache for 'brij.dev'	■ if hit
2	OS / Stub Resolver	Checks /etc/hosts and local OS cache	■ if hit
3	Router	Checks router's DNS cache (often overlooked!)	■ if hit
4	ISP Recursive NS	Your resolver checks its cache — likely a hit	■ if hit
5	Root Nameserver	Resolver asks: 'brij.dev?' → Root replies: 'Ask .dev TLD'	Stores NS
6	.dev TLD Server	Resolver asks .dev TLD → Returns brij.dev's NS records	Stores NS
7	Authoritative NS	Resolver asks brij.dev NS → Returns A record: 1.2.3.4	Stores A
8	Back to Browser	IP 1.2.3.4 returned. Browser connects via TCP/HTTPS	■ Cached

CH
3

Resolution (continued)

Caching Hierarchy

DNS caching is multi-layered. Each layer stores results up to the TTL of each record, meaning a popular domain's IP might be cached for hours at every level. Here is the performance hierarchy:



The /etc/hosts File

On Linux/macOS the file `/etc/hosts`, and on Windows `C:\Windows\System32\drivers\etc\hosts`, is checked **before** any DNS query. Entries here override DNS completely. It's the last remnant of the original HOSTS.TXT era and is still used today for:

- Blocking ads/trackers locally (e.g. `0.0.0.0 ads.example.com`)
- Development overrides (`127.0.0.1 myapp.local`)
- Testing DNS changes before propagation
- Corporate intranet hostnames

```
# /etc/hosts example
127.0.0.1    localhost
127.0.0.1    myapp.local
0.0.0.0      ads.doubleclick.net
192.168.1.50 nas.home
```

Negative Caching (NXDOMAIN)

When a domain does **not exist**, resolvers return an **NXDOMAIN** (Non-Existent Domain) response. This negative response is also cached for the duration specified by the SOA record's minimum TTL field, preventing repeated lookups for the same non-existent name. NXDOMAIN caching significantly reduces load on nameservers from typo queries.

STAT

Approx. 20-30% of all DNS queries result in NXDOMAIN — mostly from typos, misconfigured software, and browsers probing single-word address bar entries as domain names. Negative caching prevents this from overloading root servers.

CH
4

■ DNS Record Types — Complete Guide

DNS stores information as **Resource Records (RRs)**. Each record has a Name, TTL, Class (usually IN for Internet), Type, and RDATA (record-specific data). Understanding all record types is essential for domain management, email delivery, and security configuration.

Core Address Records

Type	Full Name	Purpose	Example RDATA
A	Address Record	Maps domain name to IPv4 address	142.250.80.46
AAAA	IPv6 Address Record	Maps domain name to IPv6 address	2607:f8b0:4004::200e

```
brij.dev.      300  IN  A      1.2.3.4
brij.dev.      300  IN  AAAA   2001:db8::1
```

CNAME — Canonical Name

A **CNAME record** creates an alias pointing to another domain name (not an IP). The resolver follows the chain until it finds an A or AAAA record. Important rules: you **cannot** put a CNAME at the zone apex (e.g. `brij.dev` itself), only on subdomains (`www.brij.dev`). You also cannot combine a CNAME with any other record type for the same name.

```
www.brij.dev.  3600 IN  CNAME  bij.dev.
shop.brij.dev. 3600 IN  CNAME  stores.shopify.com.
```

MX — Mail Exchange

MX records direct email for a domain to specific mail servers. Each MX record has a **priority value** (lower = higher priority). Mail servers try the lowest-priority MX first; if it fails, they try higher values. You can have multiple MX records for redundancy.

```
brij.dev. 3600 IN MX 10 mail1.brij.dev.
brij.dev. 3600 IN MX 20 mail2.brij.dev. ; backup
```

TXT — Text Record

TXT records store arbitrary text and have become the Swiss Army knife of DNS. They are used for:

Use Case	Content Pattern	Purpose
SPF	<code>v=spf1 include:_spf.google.com ~all</code>	Prevent email spoofing
DKIM	<code>v=DKIM1; k=rsa; p=MIGfMA0...</code>	Email signature verification
DMARC	<code>v=DMARC1; p=reject; rua=mailto:...</code>	Email policy enforcement
Domain Verification	<code>google-site-verification=abc123</code>	Prove domain ownership

CH
4

■ Record Types (continued)

NS — Nameserver Record

NS records specify which nameservers are authoritative for a domain. Every zone must have at least two NS records for redundancy. These are typically managed by your domain registrar or DNS hosting provider.

```
brij.dev. 86400 IN NS ns1.cloudflare.com.  
brij.dev. 86400 IN NS ns2.cloudflare.com.
```

SOA — Start of Authority

The SOA record is the most important record in a zone file — there must be exactly one per zone. It contains critical metadata about the zone itself.

```
brij.dev. IN SOA ns1.brij.dev. admin.brij.dev. (  
    2024010101 ; Serial (YYYYMMDDnn)  
    3600       ; Refresh (how often secondary checks primary)  
    900        ; Retry (if refresh fails)  
    604800     ; Expire (secondary stops answering after this)  
    300        ; Minimum TTL (negative cache TTL)  
)
```

PTR — Pointer Record

PTR records implement **reverse DNS** — mapping an IP address back to a hostname. They live in the special `in-addr.arpa` zone (IPv4) or `ip6.arpa` (IPv6). PTR records are set by the organisation that owns the IP range (usually the datacenter / ISP), not the domain owner. They are critical for email deliverability.

```
4.3.2.1.in-addr.arpa. 3600 IN PTR mail.brij.dev.
```

SRV — Service Record

SRV records specify the host and port number of servers for specific services and protocols. Format: `_service._proto.name TTL IN SRV priority weight port target`

```
_xmpp-server._tcp.brij.dev. 300 IN SRV 10 5 5269 xmpp.brij.dev.  
_sip._tls.brij.dev.         300 IN SRV 5 10 5061 sip.brij.dev.
```

CAA — Certification Authority Authorization

CAA records specify which Certificate Authorities are permitted to issue SSL/TLS certificates for your domain. This prevents rogue CAs from issuing fraudulent certificates.

```
brij.dev. 3600 IN CAA 0 issue "letsencrypt.org"  
brij.dev. 3600 IN CAA 0 issuewild ";"  
brij.dev. 3600 IN CAA 0 iodef "mailto:security@brij.dev"
```

Complete DNS Record Type Reference

Type	Full Name	RFC	Common Use
A	Address	1035	IPv4 address mapping
AAAA	IPv6 Address	3596	IPv6 address mapping
CNAME	Canonical Name	1035	Domain alias
MX	Mail Exchange	1035	Email routing
NS	Nameserver	1035	Zone delegation
TXT	Text	1035	SPF, DKIM, verification
SOA	Start of Authority	1035	Zone metadata
PTR	Pointer	1035	Reverse DNS
SRV	Service	2782	Service/port discovery
CAA	Cert. Authority Auth.	6844	SSL issuance control
NAPTR	Naming Auth. Pointer	2915	VoIP/ENUM
SSHFP	SSH Fingerprint	4255	SSH key verification
TLSA	TLS Auth.	6698	DANE / cert pinning
DNSKEY	DNS Public Key	4034	DNSSEC zone key
DS	Delegation Signer	4034	DNSSEC chain of trust
RRSIG	Resource Record Signature	4034	DNSSEC record signature
NSEC	Next Secure	4034	DNSSEC authenticated denial
HTTPS	HTTPS Svc Binding	9460	HTTPS service parameters
SVCB	Service Binding	9460	Generic service config

CH
5

TTL — Time To Live

TTL (Time To Live) is a value in seconds attached to every DNS record. It tells resolvers and clients how long to cache the record before they must re-query the authoritative nameserver. TTL is one of the most misunderstood but critically important settings in DNS management.

How TTL Works

When a resolver fetches a record with TTL=3600, it starts a countdown from 3600. Any subsequent requests within that hour are answered from cache (fast!) without touching the authoritative server. When the counter hits zero, the next request triggers a fresh lookup. Each resolver has its own TTL countdown — a record might still be cached at Resolver A while already expired at Resolver B.

TTL Value	Duration	When to Use	Tradeoff
60	1 minute	Active failover, load balancer IPs	High DNS query volume
300	5 minutes	Pre-migration (set 24h before changes)	Moderate query volume
900	15 minutes	Dynamic content / CDN edge nodes	Good balance
3600	1 hour	Standard web servers	Recommended default
21600	6 hours	Stable services	Low query volume
86400	24 hours	MX, NS records	Very low queries
604800	7 days	Rarely-changed infrastructure	Minimal queries

DNS Propagation — The Great Myth

You may have seen the phrase '*DNS changes take 24-48 hours to propagate*'. This is largely a myth. DNS changes take effect as soon as the old TTL expires — which is entirely under your control. If you set a TTL of 300 seconds (5 min) before making a change, your change will be visible to most clients within 5-10 minutes. The 24-48 hour myth comes from the days when TTLs were set to 86400 (24 hours) by default.

PRO TIP

BRIJ MOHAN'S TTL MIGRATION CHECKLIST: Step 1: 24 hours before change — lower TTL to 300s. Step 2: Make the DNS change (IP update, new CNAME, etc.). Step 3: Verify the change is working correctly. Step 4: After 24h, raise TTL back to 3600s.

CH
6

■ DNS Message Format (RFC 1035)

Every DNS query and response is a binary **DNS message** sent over UDP (port 53) or TCP (port 53 for large responses or zone transfers). Understanding the packet format is essential for network analysis and security.

DNS MESSAGE HEADER FORMAT (RFC 1035)

Transaction ID 16 bits	Flags 16 bits	Questions 16 bits	Answer RRs 16 bits	Authority RRs 16 bits	Additional RRs 16 bits
----------------------------------	-------------------------	-----------------------------	------------------------------	---------------------------------	----------------------------------

QUESTION SECTION:

QNAME: brij.dev QTYPE: A QCLASS: IN

The Flags Field (16 bits)

Bit(s)	Name	Description
15	QR	0=Query, 1=Response
14-11	Opcode	0=Standard query, 1=Inverse, 2=Status
10	AA	Authoritative Answer — set if answer is from authoritative NS
9	TC	Truncated — message exceeded 512 bytes (use TCP)
8	RD	Recursion Desired — client requests recursive resolution
7	RA	Recursion Available — server supports recursive resolution
6-4	Z	Reserved (must be 0)
3-0	RCODE	0=NoError, 1=FormErr, 2=ServFail, 3=NXDomain, 5=Refused

Common RCODE Values

RCODE	Name	Meaning	Action
0	NOERROR	Success	Use the answer
1	FORMERR	Format error — bad query	Fix query format
2	SERVFAIL	Server failure — NS couldn't process	Retry / check NS health
3	NXDOMAIN	Name doesn't exist	Domain/record doesn't exist
4	NOTIMP	Not implemented	Server doesn't support query type

5	REFUSED	Policy refusal	Access control / firewall
9	NOTAUTH	Server not authoritative for zone	Wrong nameserver queried

CH7

DNS Zones & Zone Transfers

A **DNS zone** is a portion of the DNS namespace managed by a specific organisation or person. A zone is stored in a **zone file** — a plain-text file listing all resource records for that zone. Understanding zones is fundamental to running your own nameservers.

Sample Zone File

```
; Zone file for brij.dev
$ORIGIN brij.dev.
$TTL 3600

; SOA Record
@ IN SOA ns1.brij.dev. admin.brij.dev. (
    2024040101 ; serial
    3600       ; refresh
    900        ; retry
    604800     ; expire
    300 )      ; min TTL

; Nameservers
@      IN  NS   ns1.brij.dev.
@      IN  NS   ns2.brij.dev.
ns1    IN  A    1.2.3.4      ; glue record
ns2    IN  A    1.2.3.5      ; glue record

; Web
@      IN  A    1.2.3.4
www    IN  CNAME @
api    IN  A    1.2.3.6

; Mail
@      IN  MX   10 mail.brij.dev.
mail   IN  A    1.2.3.7

; TXT
@      IN  TXT  "v=spf1 ip4:1.2.3.0/24 ~all"
```

Primary vs Secondary Nameservers

A **primary (master)** nameserver holds the original, editable copy of a zone file. A **secondary (slave)** nameserver gets a read-only copy via **zone transfer**. All zones should have at least one secondary for redundancy. ICANN requires at least two nameservers per domain.

Zone Transfers — AXFR and IXFR

Type	Name	Description	RFC
AXFR	Full Zone Transfer	Transfers entire zone file — used for initial sync	5936

IXFR	Incremental Zone Transfer	Transfers only changed records since last serial	1995
------	---------------------------	--	------

SECURITY WARNING

SECURITY: Never allow zone transfers (AXFR) from arbitrary IPs! An unrestricted AXFR leaks your entire internal network topology to attackers. Restrict AXFR to known secondary NS IPs only using TSIG (Transaction Signature) keys or IP ACLs in your DNS software.

CH

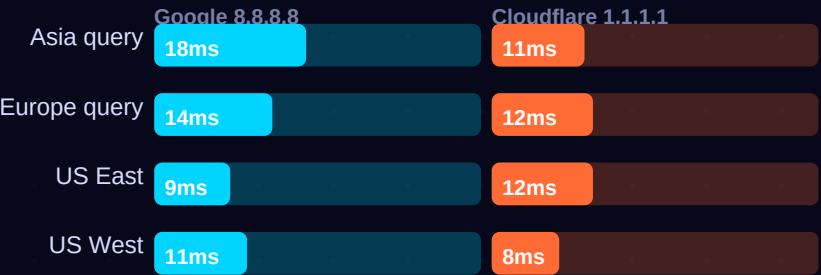
8

■

Public DNS Resolvers

Instead of using your ISP's default resolver, you can configure any device to use a public DNS resolver. Public resolvers often offer better performance, privacy, and security features.

Provider	IPv4 Primary	IPv4 Secondary	IPv6 Primary	DoH	Key Feature
Google	8.8.8.8	8.8.4.4	2001:4860:4860::8888	dns.google	Speed, reliability
Cloudflare	1.1.1.1	1.0.0.1	2606:4700:4700::1111	cloudflare-dns.com	Privacy-first
OpenDNS	208.67.222.222	208.67.220.220	2620:119:35::35	doh.opendns.com	Filtering options
Quad9	9.9.9.9	149.112.112.112	2620:fe::fe	dns.quad9.net	Malware blocking
NextDNS	Assigned	Assigned	Assigned	Custom	Fully customisable
AdGuard	94.140.14.14	94.140.15.15	2a10:50c0::ad1:ff	dns.adguard.com	Ad blocking
Comodo	8.26.56.26	8.20.247.20	—	—	Security focused



Approximate average query latency (ms) — Google 8.8.8.8 vs Cloudflare 1.1.1.1

Cloudflare 1.1.1.1 — Privacy Focus

Cloudflare's 1.1.1.1 launched in April 2018 and quickly became the world's fastest public DNS resolver. Key differentiators: they do not log queries for longer than 24 hours, do not sell user data, support DNS-over-HTTPS and DNS-over-TLS natively, and offer 1.1.1.2 (malware blocking) and 1.1.1.3 (malware + adult content blocking) as optional variants.

CH
9

■ DNS Security — Attack Types

DNS was designed for correctness and scalability, not security. Its age and ubiquity make it a prime target for attackers. Understanding these attacks is the first step to defending against them.

1. DNS Cache Poisoning

An attacker sends forged DNS responses to a resolver, attempting to replace a legitimate cached record with a malicious IP. If successful, every client using that resolver gets redirected to the attacker's server. The **Kaminsky attack (2008)** showed this could be done in seconds using birthday-paradox techniques against the 16-bit transaction ID.

MITIGATION

MITIGATION: Source Port Randomisation (RFC 5452) + Transaction ID randomisation vastly increases the difficulty. DNSSEC eliminates it entirely by cryptographically verifying every response.

2. DNS Spoofing / Man-in-the-Middle

Similar to cache poisoning but targeted at a specific client rather than a resolver. An attacker on the same network (coffee shop, hotel Wi-Fi) intercepts DNS queries and replies with forged responses. The victim has no way to detect this without DNSSEC or DoH/DoT.

3. DNS Hijacking

DNS hijacking can occur at multiple levels: at the router (attacker changes DNS settings on compromised router), at the ISP (ISP redirects failed queries to ad-laden search pages), or nationally (governments redirect political sites to censored pages). Some malware families specifically target DNS settings.

4. DNS Amplification / DDoS

DNS amplification exploits two properties: DNS queries are tiny (30-60 bytes) but responses can be huge (up to 65KB), and DNS uses UDP where the source IP can be spoofed. Attackers send tiny queries with the victim's IP as the source to thousands of open DNS resolvers — all the massive responses flood the victim.

Aspect	Value
Query size	~60 bytes
ANY response size	~3,000 bytes
Amplification factor	~50x
Mirai botnet attack	1.2 Tbps peak (2016)
Mitigation	BCP38 ingress filtering, Response Rate Limiting (RRL)

■ DNS Attacks (continued)

5. DNS Tunneling

DNS tunneling abuses the DNS protocol to exfiltrate data or create covert command-and-control (C2) channels. Because DNS is almost always allowed through firewalls, attackers encode data in DNS query names (aGVsbG8.attacker.com → 'hello' in base64) and extract responses from TXT records. This technique is used by both APTs (Advanced Persistent Threats) and malware families like FrameworkPOS.

Detection methods include: monitoring for unusually long or high-entropy DNS query names, high query rates to a single domain, unusual query types (especially TXT), and anomalous DNS traffic volumes.

6. NXDOMAIN Hijacking

Some ISPs and resolvers intercept NXDOMAIN responses (non-existent domains) and instead return the IP of an ad-laden search page. This breaks applications that rely on NXDOMAIN as a signal and is considered a violation of DNS standards by ICANN.

7. Domain Squatting & Typosquatting

Attackers register domains that are visually similar to legitimate ones: paypa1.com (number 1 instead of letter l), microsoft.com, or Unicode lookalike characters in IDN domains. Used for phishing, credential theft, and malware distribution.

Attack Summary & Mitigations

Attack	Impact	Primary Mitigation
Cache Poisoning	■ HIGH	DNSSEC + Source port randomisation
DNS Spoofing	■ HIGH	DoH / DoT + DNSSEC
DNS Hijacking	■ MED	DoH, trusted resolver, monitor changes
DDoS Amplification	■ HIGH	BCP38 filtering, RRL, anycast
DNS Tunneling	■ MED	Deep packet inspection, DNS monitoring
NXDOMAIN Hijacking	■ LOW	Use reputable resolver (1.1.1.1, 8.8.8.8)
Typosquatting	■ MED	Register similar domains, user education
Zone Transfer Leak	■ HIGH	Restrict AXFR, use TSIG keys

CH
10

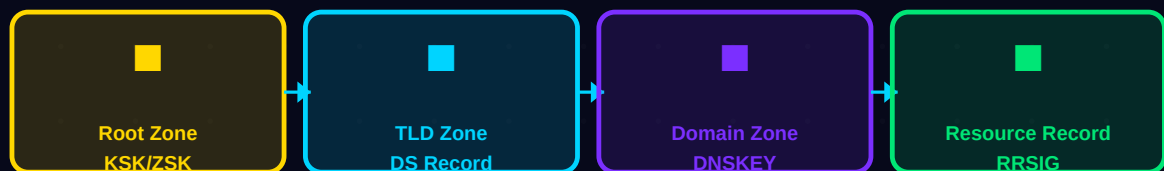
■ DNSSEC — DNS Security Extensions

DNSSEC (DNS Security Extensions) adds cryptographic authentication to DNS. It allows resolvers to verify that DNS responses are authentic (from the real authoritative server) and have not been tampered with. DNSSEC does **not** encrypt DNS traffic — it only authenticates it. For encryption, you need DoH or DoT.

How DNSSEC Works

DNSSEC uses public-key cryptography to sign zone data. Each zone has two key pairs: the **Key Signing Key (KSK)** — used to sign the DNSKEY record set — and the **Zone Signing Key (ZSK)** — used to sign all other records. The KSK's public key fingerprint (a DS record) is stored in the parent zone, creating a chain of trust.

DNSSEC CHAIN OF TRUST — Each level signs the next



DNSSEC Record Types

Record	Purpose	Who Sets It?
DNSKEY	Stores the zone's public signing keys (KSK + ZSK)	Zone owner
RRSIG	Cryptographic signature over each RRset	Zone owner (auto)
DS	Hash of child zone's KSK — stored in parent zone	Registrar uploads to TLD
NSEC	Proves a name/type does NOT exist (authenticated denial)	Zone owner (auto)
NSEC3	Like NSEC but hashes names (prevents zone enumeration)	Zone owner (auto)
CDS/CDNSKEY	Child-initiated DS/DNSKEY update automation (RFC 8078)	Zone owner

CH10

DNSSEC (continued)

DNSSEC Validation Process

When a DNSSEC-validating resolver receives a response, it performs the following steps: 1) Retrieves the DNSKEY records for the zone. 2) Uses the RRSIG to verify the answer records are signed by the ZSK. 3) Verifies the DNSKEY (ZSK) is signed by the KSK. 4) Fetches the DS record from the parent zone. 5) Verifies the DS record matches a hash of the KSK. 6) Continues up to the root (which is self-signed and included in all resolvers). Only if the entire chain validates does the resolver trust the answer.

DNSSEC Adoption Statistics

As of 2024, DNSSEC is signed on approximately 25% of all .com domains and over 90% of government domains in Sweden and the Netherlands. The root zone has been signed since 2010. Major resolvers including Google 8.8.8.8, Cloudflare 1.1.1.1, and Comcast all validate DNSSEC.

Domain Category	DNSSEC Signed %
.gov (US Federal)	~100%
.se (Sweden)	~60%
.nl (Netherlands)	~60%
.com (generic)	~25%
.org (generic)	~20%
Root zone	100% (since 2010)

OPERATIONAL NOTE

DNSSEC adoption has been slow due to operational complexity. A misconfigured DNSSEC setup causes complete DNS failure for a domain — often worse than no security at all. Many registrars now offer one-click DNSSEC activation that handles key management automatically.

CH
11

■ Modern DNS — DoH, DoT, DoQ

Traditional DNS sends queries in **plaintext over UDP port 53**. This means anyone between you and the resolver (your ISP, router, network attacker) can see exactly what domains you visit — a significant privacy leak. Modern encrypted DNS protocols address this.

DNS over HTTPS (DoH) — RFC 8484

DoH sends DNS queries as standard HTTPS requests to a resolver's API endpoint. Because it uses port 443 and looks identical to regular HTTPS traffic, it is very difficult to block or monitor. Used natively by Firefox, Chrome, Edge, and Android.

```
# DoH query example using curl
curl -s 'https://cloudflare-dns.com/dns-query?name=brij.dev&type=A' \
-H 'Accept: application/dns-json'
```

DNS over TLS (DoT) — RFC 7858

DoT wraps DNS queries in a TLS tunnel on **port 853**. Unlike DoH it's clearly identifiable as DNS (port 853 is reserved for DoT), which makes it easier for network admins to monitor or block if needed. Supported by Android 9+ (Private DNS), iOS 14+, and major resolvers.

DNS over QUIC (DoQ) — RFC 9250

DoQ is the newest protocol, running DNS over QUIC (the same transport protocol behind HTTP/3). QUIC provides lower latency than TLS because it eliminates the TCP handshake, has built-in connection multiplexing, and handles packet loss more gracefully. DoQ uses UDP port 853.

Feature	Classic DNS	DoH	DoT	DoQ
Port	53 (UDP/TCP)	443 (TCP)	853 (TCP)	853 (UDP)
Transport	UDP/TCP	TLS over TCP	TLS/TCP	QUIC
Encrypts queries?	No	Yes	Yes	Yes
Hides from ISP?	No	Yes	Partial	Yes
Network admin visible?	Yes	No	Yes (port)	Partial
Browser native?	Yes	Yes	No	Emerging
Latency	Low	Medium	Medium	Low
RFC	1034/1035	8484	7858	9250
Year standardised	1987	2018	2016	2022

CH
11

■ Modern DNS (continued)

Configuring DoH on Major Platforms

Platform	Where to Configure	DoH Support
Firefox	Settings → Privacy → DNS over HTTPS	Built-in since v60
Chrome	Settings → Privacy → Use secure DNS	Built-in since v83
Edge	Settings → Privacy → Use secure DNS	Built-in since 2020
Windows 11	Network Settings → DNS encryption	Native OS since 2021
Android 9+	Settings → Network → Private DNS	DoT (not DoH)
iOS 14+	Configuration profile or 3rd party app	DoH + DoT
Linux (systemd-resolved)	/etc/systemd/resolved.conf	DoT native
macOS 11+	Requires 3rd party app or config profile	DoH + DoT

The Privacy vs. Control Debate

DoH has sparked significant controversy. ISPs and enterprise IT teams argue that encrypting DNS undermines legitimate network monitoring, parental controls, corporate security filtering, and law enforcement capabilities. Privacy advocates argue users have a fundamental right to private browsing. Most browsers implement a 'canary domain' check — if your ISP blocks that domain, DoH is disabled to respect enterprise policies.

OPINION: BRIJ MOHAN

Brij Mohan's take: For personal devices on public or home networks, enabling DoH significantly improves privacy. For corporate environments, deploy an internal DoH resolver so you get encryption benefits while maintaining visibility. Never blanket-block DoH — it encourages users to bypass controls.

CH
12

Troubleshooting & Tools

DNS issues are among the most common causes of website outages, email delivery failures, and network connectivity problems. Knowing the right tools and methodology saves hours of debugging.

Essential DNS Tools

Tool	Platform	What it does	Best For
dig	Linux/macOS	Full-featured DNS query tool with raw output	Deep debugging, scripting
nslookup	All platforms	Interactive DNS lookup tool	Quick checks, Windows
host	Linux/macOS	Simple DNS lookup, human-readable	Quick lookups
drill	Linux	DNSSEC-aware lookup (like dig but cleaner)	DNSSEC validation
whois	All platforms	Domain registration information	Registrar/ownership info
mtr	Linux/macOS	Combined ping + traceroute (finds DNS hops)	Network path analysis
Wireshark	All platforms	Full packet capture including DNS traffic	Deep packet analysis
doggo	All platforms	Modern dig alternative with DoH/DoT support	Modern DNS queries
dnscrypt-proxy	All platforms	Local proxy for DoH/DoT/DNSCrypt	Privacy-focused setup
nmap --script	All platforms	DNS enumeration and zone transfer testing	Security auditing

CH
12

■ Troubleshooting (continued)

The dig Command — Complete Reference

```
# Basic A record lookup
dig brij.dev A

# Use specific DNS server
dig @8.8.8.8 brij.dev A

# Query all record types
dig brij.dev ANY

# Get short answer only
dig brij.dev A +short

# Show TTL and authority section
dig brij.dev A +ttlid +authority

# Trace the full resolution path
dig brij.dev A +trace

# DNSSEC validation check
dig brij.dev DNSKEY +dnssec

# Reverse lookup (PTR)
dig -x 1.2.3.4

# Query via DoH
dig @https://cloudflare-dns.com/dns-query brij.dev A

# Check MX records
dig brij.dev MX +short

# Test zone transfer (should fail if secured)
dig @ns1.brij.dev brij.dev AXFR
```

Troubleshooting (continued)

Common DNS Problems & Solutions

Problem	Symptoms	Diagnosis	Solution
Wrong IP in A record	Site unreachable after dig domain.com A	dig domain.com A	Update A record, wait TTL
MX record missing	Email bouncing	dig domain.com MX	Add/fix MX record
SPF failing	Emails going to spam	dig domain.com TXT	Fix/add SPF TXT record
CNAME at zone apex	Site not loading	dig domain.com CNAME	Use A record at apex
NS records wrong	All DNS failing	dig domain.com NS	Fix at registrar
TTL too high	Changes not propagating	dig +ttl domain.com A	Lower TTL before changes
NXDOMAIN for valid domain	Domain not found	dig @8.8.8.8 domain.com	Check NS, zone propagation
SERVFAIL	Intermittent failures	dig +trace domain.com	Check NS health, DNSSEC
DNSSEC validation failure	SERVFAIL on validating	dig +dnssec domain.com	Fix DNSSEC keys/DS record
PTR record missing	Email marked as spam	dig -x <your-mail-IP>	Ask hosting for PTR record
DNS propagation seems slow	Some users see old IP	dig @multiple-resolvers	Wait for TTL expiry
DNS hijacking by ISP	Wrong IP returned	dig @8.8.8.8 vs @ISP-NS	Use trusted resolver (DoH)

Online DNS Tools

Tool	URL	Purpose
MXToolbox	mxtoolbox.com	Email DNS (MX, SPF, DKIM, DMARC)
DNS Checker	dnschecker.org	Global propagation checker
WhatsMyDNS	whatsmydns.net	DNS propagation worldwide
DNSViz	dnsviz.net	DNSSEC chain visualiser
Dig (web)	toolbox.googleapps.com	Google dig web interface
SSL Labs DNS Test	ssllabs.com	SSL + DNS combined audit
Cloudflare Radar	radar.cloudflare.com	Global DNS trends and health
RIPE Atlas	atlas.ripe.net	Distributed DNS measurement

CH
0

■ Summary & Key Takeaways

What You've Learned

Topic	Key Lesson
■ History	DNS replaced HOSTS.TXT in 1983. BIND still powers much of the internet.
■ Architecture	3-tier hierarchy: Root → TLD → Authoritative. All distributed, no central point.
■ Resolution	8-step resolution process. Most answers served from cache in <5ms.
■ Records	20+ record types. Master A, CNAME, MX, TXT, SOA, NS, PTR, SRV, CAA.
■ TTL	You control propagation speed. Lower TTL before migrations.
■ Protocol	Binary UDP protocol. 13 root servers. RCODE tells you what went wrong.
■ Zones	Zone files hold all records. Restrict AXFR to prevent leaks.
■ Resolvers	1.1.1.1 and 8.8.8.8 for best performance. Consider privacy needs.
■ Attacks	Cache poisoning, tunneling, amplification, hijacking — all real threats.
■ DNSSEC	Cryptographic chain of trust. Adopt it. Automate key management.
■ Modern DNS	DoH/DoT/DoQ encrypt your queries. Enable on personal devices.
■ Tools	dig, nslookup, Wireshark, MXToolbox. Master these for fast debugging.

■ ABOUT THE AUTHOR

Brij Mohan is a network engineer and web technologist with deep expertise in DNS infrastructure, security, and pro